BTEUP EXAM NOTES | HINGLISH EDITION

# Mobile App Development

## Unit I: Introduction to Android: Platform, SDK, First App, Manifest

**In notes ke baare me**

**Part A** me har topic simple Hinglish me real-life example ke saath samjhaya hai, taaki beginner bhi zero se samajh sake.

**Part B** me important exam questions ke poore 5-marks wale answers Hinglish me diye hain.

Last me: quick revision sheet, common mistakes aur self-test.

*Exam me answer English me likhna hai to English Edition PDF use karo.*

**Syllabus covered (Unit I)**

Introduction to Android: The Android Platform, Android SDK, Eclipse Installation, Android Installation, Building your First Android application, Understanding Anatomy of Android Application, Android Manifest file.

# Unit Overview aur Contents

**Simple idea:** Android zyada tar mobile phones ka operating system hai. Iske liye app banane ke liye jaanna padta hai: (1) **Android kaise bana hai** (platform), (2) **tools** (SDK aur IDE), (3) computer aur phone ko **kaise set up** karein, (4) **pehla app kaise banayein**, aur (5) **app me kaunsi files hoti hain**, khaas kar **AndroidManifest.xml**, jo app ka ID card hai.

**Eclipse ke baare me zaroori note**

Tumhare syllabus me **Eclipse** likha hai, isliye Eclipse ke steps diye hain. Par Google ne **Eclipse ADT (Android Developer Tools) ka official support 2016 me band kar diya**. Ab **Android Studio** official IDE hai. Isliye in notes me **dono** hain: Eclipse + ADT (exam ke liye, syllabus ke hisab se) aur Android Studio (jo practice me use karna chahiye).

| Topic | Exam importance | Notes me kahan |
|---|---|---|
| Android platform aur architecture | Very High | Part A: Topic 1 \| Part B: Q1, Q2, Q12 |
| Android SDK | Very High | Part A: Topic 2 \| Part B: Q3 |
| Eclipse aur Android installation | High | Part A: Topic 3, 4 \| Part B: Q4, Q5, Q6 |
| Pehla Android application | Very High | Part A: Topic 5 \| Part B: Q7 |
| Android application ki anatomy | Very High | Part A: Topic 6 \| Part B: Q8, Q9, Q11 |
| AndroidManifest.xml | Very High | Part A: Topic 7 \| Part B: Q10 |

## Contents


| Part A: Easy Explanation | Part B: Important Questions aur Answers |
|---|---|
| Topic 1: The Android Platform<br>Topic 2: Android SDK<br>Topic 3: Eclipse aur Android Studio installation<br>Topic 4: Android installation (emulator, phone)<br>Topic 5: Pehla Android application<br>Topic 6: Android application ki anatomy<br>Topic 7: AndroidManifest.xml | Q1 Android: features and versions (Very Likely)<br>Q2 Android architecture (Very Likely)<br>Q3 Android SDK (Very Likely)<br>Q4 Android Studio installation (Likely)<br>Q5 Eclipse + ADT installation (Likely)<br>Q6 AVD, emulator, run on phone (Likely)<br>Q7 First Android application (Very Likely)<br>Q8 Anatomy of Android application (Very Likely)<br>Q9 Build process, R.java, resources (Likely)<br>Q10 AndroidManifest.xml (Very Likely)<br>Q11 Components of an app (Likely)<br>Q12 Dalvik vs ART, APK vs AAB (Possible) |


**Is unit ko kaise padhein (time: 1 din)**

1) Part A ek baar padh kar samjho. 2) Architecture diagram, project structure aur manifest ko bina dekhe banane tak practice karo. 3) Quick Revision Sheet se revise karo. 4) Self-Test bina dekhe karo.

# PART A: Easy Explanation (Hinglish)

## Topic 1: The Android Platform

**Real-life example**

Smartphone ek restaurant jaisa hai. **Operating system (Android)** kitchen aur manager hai jo sab kuch chalata hai. **Apps** wo dishes hain jo customer order karta hai. Nayi dishes (apps) banane ke liye kitchen ka kaam samajhna padta hai.

**Definition**

**Android** ek open-source, Linux-based mobile operating system hai, jo **Google** aur **Open Handset Alliance (OHA)** ne develop kiya hai, aur smartphones, tablets, smart TVs, watches aur cars me use hota hai.

### Quick history

- **2003:** Android Inc. bani (Andy Rubin aur anya). **2005:** Google ne ise khareed liya.
- **2007:** Open Handset Alliance bana. **2008:** pehla Android phone, **HTC Dream** (Android 1.0).
- Android versions ke naam **desserts** par the (Cupcake, Donut ... Pie). Android 10 se sirf numbers use hote hain.

| Version | Name | API level |
|---|---|---|
| 1.5 | Cupcake | 3 |
| 2.2 | Froyo | 8 |
| 2.3 | Gingerbread | 9-10 |
| 4.0 | Ice Cream Sandwich | 14-15 |
| 4.1 - 4.3 | Jelly Bean | 16-18 |
| 4.4 | KitKat | 19 |
| 5.0 - 5.1 | Lollipop (ART becomes default) | 21-22 |
| 6.0 | Marshmallow (runtime permissions) | 23 |
| 7.0 - 7.1 | Nougat | 24-25 |
| 8.0 - 8.1 | Oreo | 26-27 |
| 9 | Pie | 28 |
| 10 / 11 / 12 | (no dessert name) | 29 / 30 / 31 |
| 13 / 14 / 15 / 16 | (no dessert name) | 33 / 34 / 35 / 36 |

**Yaad rakho**

**API level** wo number hai jo developers ke liye Android platform version ko pehchanta hai (jaise Android 5.0 = API 21). Tumhara app sabse kam kaunsa API level support karta hai wo batata hai (**minSdk**). Sabse naye release ke liye developer.android.com dekho.

### Android ke features

- **Open source aur free:** manufacturers ise use aur customise kar sakte hain.
- Widgets, notifications, split screen ke saath **rich UI aur multitasking**.
- **Connectivity:** Wi-Fi, Bluetooth, NFC, GPS, 4G/5G.

- **Storage:** SQLite database, files, shared preferences.
- **Hardware access:** camera, sensors, microphone, fingerprint.
- **Apps Java ya Kotlin me** (Google Kotlin ko prefer karta hai), Google Play se distribute hote hain.
- **Security:** har app apne sandbox me chalta hai aur permissions maangta hai.

### Android Architecture (Platform stack)

Android **layers** me bana hai. Har layer apne neeche wali layer ki services use karti hai. Ye ek multi-storey building jaisa hai: foundation Linux kernel hai aur sabse upar apps hain.

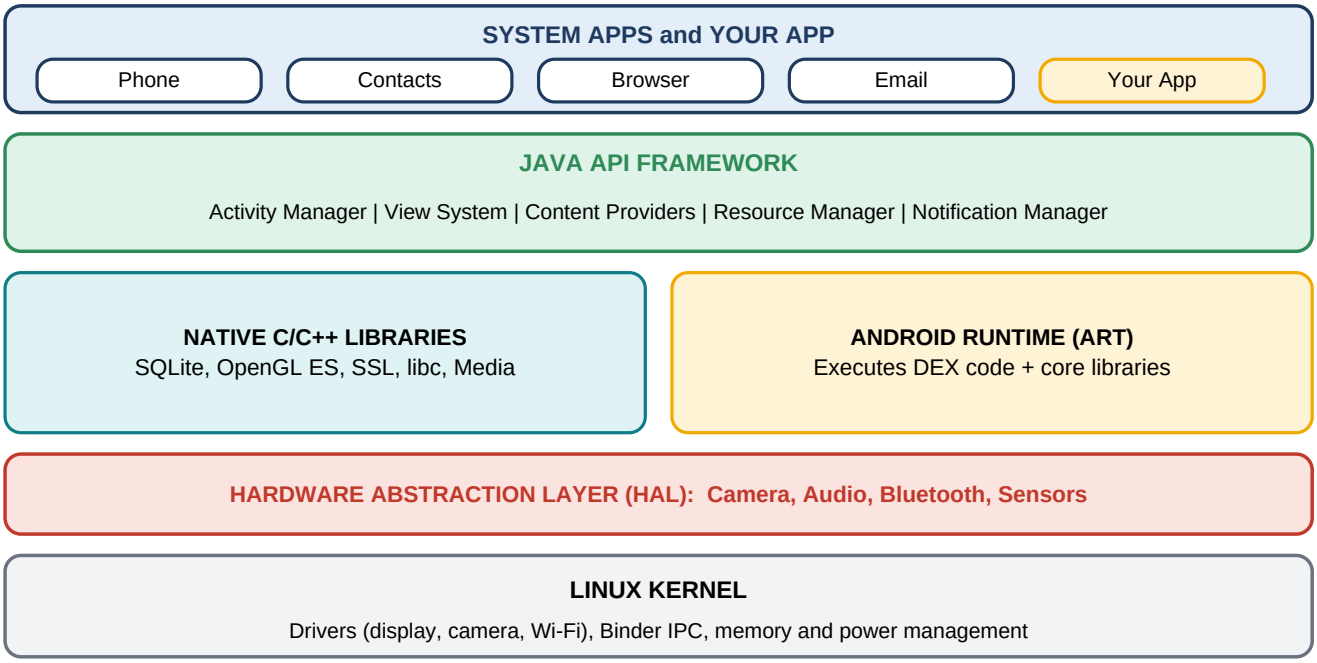



*Figure 1: Android software stack (architecture)*

| Layer (upar se neeche) | Ye kya karti hai |
|---|---|
| **System apps** | Built-in apps (phone, contacts, browser, email). Tumhara app bhi isi level par hota hai. |
| **Java API framework** | Wo APIs jinse app banta hai: Activity Manager (life cycle), View System (UI), Content Providers (data share), Resource Manager, Notification Manager. |
| **Native C/C++ libraries** | C/C++ me likhi core libraries jaise SQLite (database), OpenGL ES (graphics), SSL (security), libc, media libraries. |
| **Android Runtime (ART)** | App ka DEX code chalata hai; har app apne process me apne ART instance ke saath chalta hai. Core Java libraries bhi hoti hain. Android 5.0 se pehle runtime Dalvik tha. |
| **Hardware Abstraction Layer (HAL)** | Hardware (camera, Bluetooth, audio, sensors) ke liye standard interfaces deti hai, taaki upar ki layers ko hardware details na jaanni pade. |
| **Linux kernel** | Foundation: device drivers, memory, process aur power management, security, networking. |

**Textbook note**

Kai textbooks 4-layer view dikhate hain: **Applications, Application Framework, Libraries + Android Runtime, Linux Kernel**. Ye wahi stack hai. Google ki current documentation HAL ko alag layer dikhati hai. Agar tumhari textbook me 4 layers hain to wahi banao aur HAL ka mention kar do.

## Topic 2: Android SDK (Software Development Kit)

**Real-life example**

Furniture banane ke liye tools chahiye: aari, hathoda, drill. **Android SDK** Android apps banane ka **toolbox** hai: isme libraries, tools aur test karne ke liye ek virtual phone hota hai.

**Definition**

**Android SDK** libraries, tools, documentation aur sample code ka collection hai, jo developers ko Android applications **build, test aur debug** karne ke liye chahiye.

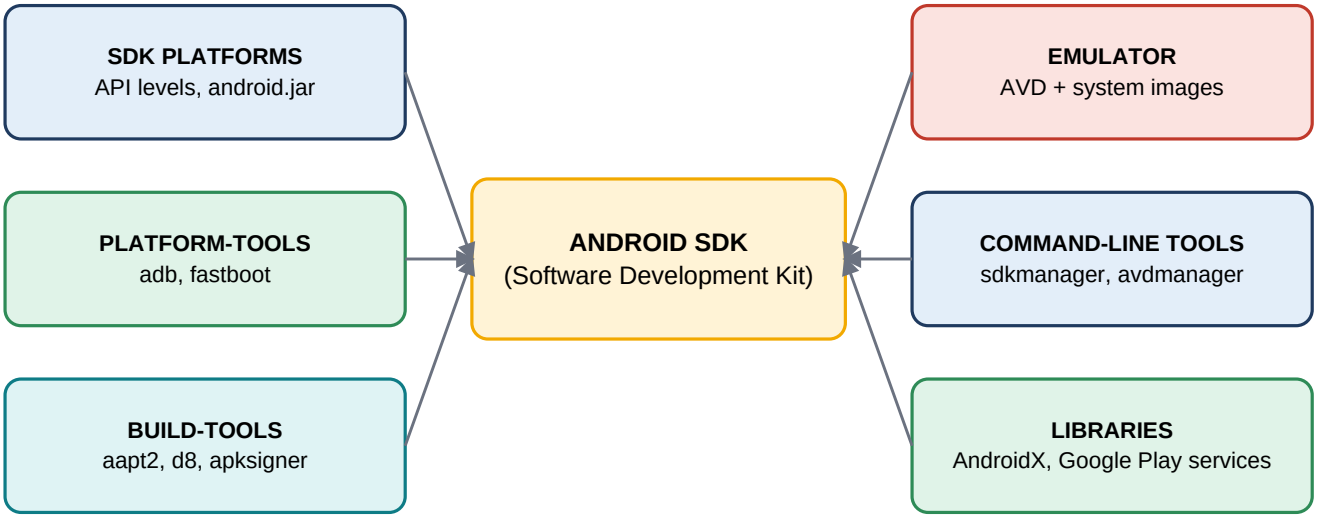



*Figure 2: Main components of the Android SDK*

| Component | Purpose |
|---|---|
| **SDK Platforms** | Har API level ki android.jar library; tum app ko ek platform ke against compile karte ho |
| **Platform-Tools** | Device se baat karne ke liye **adb** (Android Debug Bridge), flashing ke liye **fastboot** |
| **Build-Tools** | **aapt2** (resources), **d8** (DEX), **apksigner**, **zipalign** |
| **Emulator and system images** | System images (jaise Android 14 x86_64) se bane virtual phones (AVD) |
| **Command-line tools** | sdkmanager (packages install), avdmanager (AVDs banana) |
| **SDK Manager** | IDE ki window jisse platforms, tools aur images download hote hain |
| **Libraries** | AndroidX / Jetpack libraries, Google Play services |

- **Aur chahiye: JDK** (Java Development Kit). Android Studio apna JDK saath laata hai; Eclipse ke liye JDK khud install karna padta hai.
- **Logcat** debugging ke liye system aur app log messages dikhata hai. **DDMS** Eclipse ka purana debugging tool tha; uske features ab Android Studio me hain.
- **Key SDK settings: compileSdk** (jis API ke against compile), **minSdk** (sabse purana Android jis par app chalega), **targetSdk** (jis version ke liye test kiya).

## Topic 3: Installation of the Development Environment

### 3(a) Android Studio installation (current, official IDE)

**Real-life example**

Android Studio ek poori tarah equipped workshop hai ek box me: editor, SDK, emulator aur build tools saath aate hain.

- **Step 1: developer.android.com/studio** par system requirements dekho (64-bit OS, kam se kam 8 GB RAM jisme 16 GB recommended, kai GB free disk, emulator ke liye hardware virtualization on).
- **Step 2:** Apne OS ke liye Android Studio installer download karo (Windows .exe, macOS .dmg, Linux .tar.gz).
- **Step 3:** Installer chalao. **Android Studio** aur **Android Virtual Device** selected rakho, install folder chuno aur Install dabao.
- **Step 4:** Pehli baar kholne par **Setup Wizard** khulta hai. **Standard** setup chuno, theme chuno aur licences accept karo.
- **Step 5:** Wizard **SDK, platform-tools aur emulator** download karta hai. Khatam hone tak ruko aur Finish dabao.
- **Step 6:** Kabhi bhi **Tools > SDK Manager** khol kar aur platforms ya tools install karo.

### 3(b) Eclipse installation (as per syllabus; now discontinued)

**'Discontinued' kyun?**

Google ne 2015 me announce kiya ki Eclipse ADT support band hoga aur Android Studio 2.2 ke baad 2016 me formally band kar diya. Nayi SDK features Eclipse me kaam nahi karte. Ye steps exam ke liye seekho, par real kaam ke liye Android Studio use karo.

- **Step 1: JDK install karo** aur **JAVA_HOME** environment variable set karo.
- **Step 2: Eclipse download aur unzip karo** (Eclipse IDE for Java Developers). Installer nahi chahiye; eclipse.exe chalao.
- **Step 3: Android SDK download karo** (standalone SDK Tools) aur **SDK Manager** khol kar platform (jaise Android 4.x), platform-tools aur build-tools install karo.
- **Step 4: ADT plugin install karo: Help > Install New Software > Add**, naam (ADT) aur ADT update-site URL daalo, **Developer Tools** select karo, licence accept karo aur Eclipse restart karo.
- **Step 5: Eclipse ko SDK ka path do: Window > Preferences > Android**, **SDK Location** set karo.
- **Step 6: AVD banao: Window > AVD Manager > New**.
- **Step 7: Project banao: File > New > Android Application Project**.

## Topic 4: Android Installation (running Android on an emulator or a phone)

Yahan 'Android installation' ka matlab hai **apna app chalane aur test karne ke liye Android device ready karna**: ya to virtual device (emulator) ya real phone.

### 4(a) Create an AVD (Android Virtual Device) and use the emulator

> **Real-life example**
> Emulator tumhare computer ke andar ek **practice phone** hai. App test karne ke liye doosra phone kharidne ki zaroorat nahi.

- **1.** Android Studio me **Tools > Device Manager** kholo aur **Create Device** dabao.
- **2.** Ek **hardware profile** chuno (jaise Pixel) aur Next dabao.
- **3.** Ek **system image** chuno (Android version/API level; PC par x86_64 images tez hoti hain). Zaroorat ho to download karo.
- **4.** AVD ko naam do, settings dekho aur **Finish** dabao.
- **5.** Emulator shuru karne ke liye AVD ke saamne **Play** button dabao. Achhi speed ke liye BIOS me hardware acceleration (virtualization) on honi chahiye.

### 4(b) Run the app on a real Android phone

- **1. Developer options on karo:** Settings > About phone > **Build number** par 7 baar tap karo.
- **2. USB debugging on karo:** Settings > Developer options > **USB debugging**.
- **3. Phone ko USB cable se connect karo** aur 'Allow USB debugging?' prompt par **Allow** dabao.
- **4. Connection check karo:** `adb devices` chalao; tumhara device list me dikhna chahiye.
- **5. Run karo:** device drop-down me phone chuno aur green **Run** button dabao.

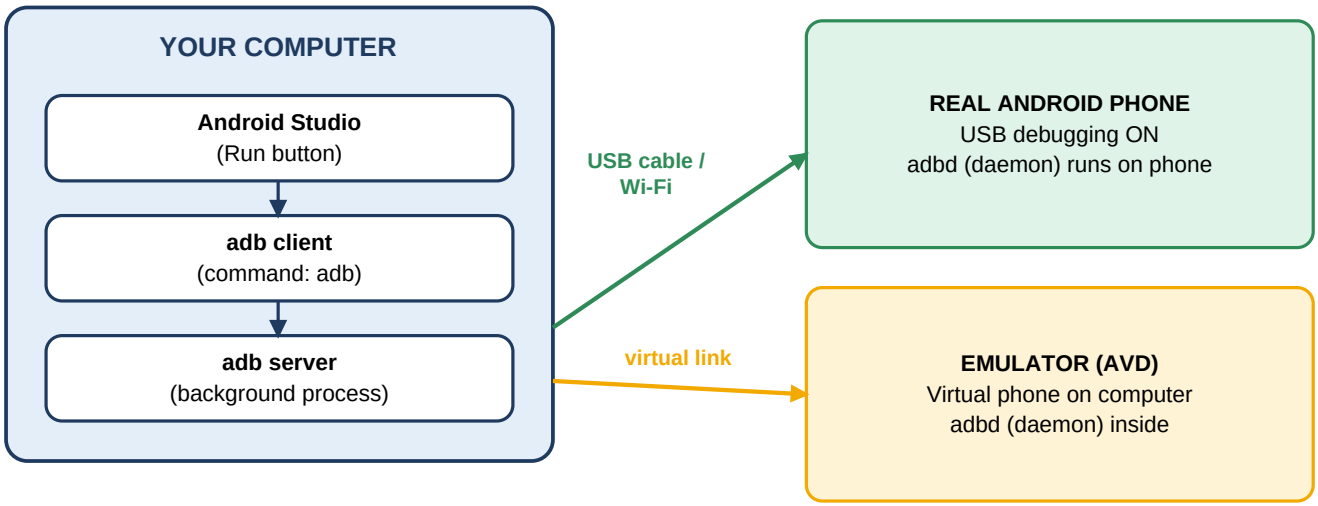



*Figure 3: How the IDE talks to a device or emulator through adb*

### 4(c) Installing an APK directly

- **APK** (Android Package) install hone wali app file hai. Isse install karo: `adb install app-debug.apk`.
- Phone par APK copy karke khol bhi sakte ho, us file manager ke liye **Install unknown apps** allow karne ke baad. Sirf trusted sources se apps install karo.

## Topic 5: Building your First Android Application (Hello World)

**Real-life example**

Har language ka pehla program 'Hello World' print karta hai. Ye prove karta hai ki tools sahi install hue hain aur app ke basic parts dikhata hai.

### Android Studio me steps

- **1.** Android Studio kholo aur **New Project** chuno (ya File > New > New Project).
- **2. Phone and Tablet** tab aur **Empty Views Activity** template chuno (purane versions me 'Empty Activity' kehte the). Next dabao.
- **3. Name** (HelloWorld), **Package name** (com.example.helloworld), save location, **Language** (Java ya Kotlin) aur **Minimum SDK** daalo.
- **4. Finish** dabao. **Gradle** project build kare tab tak ruko (pehla sync kuch minute le sakta hai).
- **5. res/layout/activity_main.xml** (screen) aur **MainActivity** (code) kholo.
- **6.** Device (emulator ya phone) chuno aur green **Run** button dabao. Screen par 'Hello World!' dikhta hai.

**MainActivity.java**

```
package com.example.helloworld;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

**res/layout/activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

### Code ko line by line samjho

| Line | Matlab |
|---|---|
| `package com.example.helloworld;` | Package name: tumhare code ka unique address. |
| `import ...AppCompatActivity;` | Activities ki base class laata hai (AppCompatActivity modern look aur backward compatibility deti hai). |
| `class MainActivity extends AppCompatActivity` | Hamari screen ek **Activity** hai. 'extends' ka matlab wo AppCompatActivity ke features inherit karti hai. |
| `@Override` | Hum parent class ke ek method ko replace (override) kar rahe hain. |
| `onCreate(Bundle savedInstanceState)` | Activity pehli baar bante samay call hota hai. Bundle saved state rakhta hai (jaise rotation ke baad). |
| `super.onCreate(savedInstanceState);` | Pehle parent class ka setup code chalata hai. Ye line zaroori hai. |
| `setContentView(R.layout.activity_main);` | activity_main.xml layout file ko screen ki tarah dikhata hai. R.layout.activity_main us file ka ID hai. |
| `layout_width / layout_height` | **match_parent** = parent jitna bada; **wrap_content** = sirf content jitna bada. |
| `android:text="Hello World!"` | TextView jo text dikhata hai. |

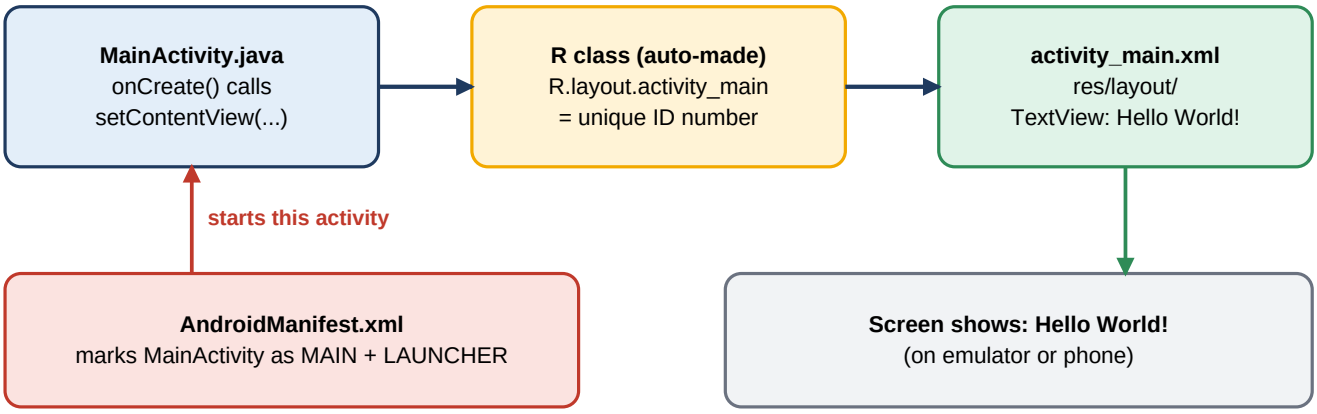



*Figure 4: How the Manifest, MainActivity, R class and layout work together*

**Purane (Eclipse) projects**

Eclipse projects me activity `Activity` (ya ActionBarActivity) ko extend karti thi aur layout `RelativeLayout` use karta tha. Idea bilkul same hai: `onCreate()` `setContentView(R.layout.activity_main)` ko call karta hai.

## Topic 6: Anatomy of an Android Application

> **Real-life example**
> Insaan ki anatomy shareer ke parts (dil, dimaag, haddiyan) dikhati hai. **Android app ki anatomy** project ke parts dikhati hai: code, screens, images aur settings kahan rehte hain.

### Android Studio project structure (Android view)

```
app                                   Main module of your app
  |-- manifests
      `-- AndroidManifest.xml         Declares components, permissions, app info
  |-- java
      `-- com.example.helloworld      Package name
          `-- MainActivity            Java / Kotlin source code (logic)
  `-- res
      |-- drawable                    Images and shapes
      |-- layout                      UI screens (activity_main.xml)
      |-- mipmap                      Launcher icons
      `-- values                      strings.xml, colors.xml, themes.xml
Gradle Scripts                        Build configuration
  |-- build.gradle (Project)          Settings for the whole project
  |-- build.gradle (Module: app)      App id, minSdk, targetSdk, dependencies
  |-- settings.gradle                 Lists the modules
  |-- gradle.properties               Gradle options
  `-- local.properties                Local SDK path (not shared)
```

*Figure 5: Project structure in Android Studio*

| Folder / file | Isme kya hota hai |
|---|---|
| **manifests / AndroidManifest.xml** | App ka ID card: package, components, permissions, icon, theme |
| **java** (or kotlin) | Source code: activities, classes. Test folders bhi. |
| **res/drawable** | Images, icons aur shape XML files |
| **res/layout** | Har screen design karne wali XML files |
| **res/mipmap** | Alag sizes ke launcher icons |
| **res/values** | strings.xml (text), colors.xml, themes.xml (purana: styles.xml), dimens.xml |
| **Gradle Scripts** | Build files. **build.gradle (Module: app)** me: applicationId, minSdk, targetSdk, versionCode, versionName, dependencies |

### Classic Eclipse project structure (purani textbooks jaisa)

| Folder / file | Purpose |
|---|---|
| **src** | Java source files (MainActivity.java) |
| **gen** | Auto-generated files, mukhya taur par **R.java**. Ise kabhi edit mat karo. |
| **Android x.x** | Chune hue platform ki android.jar library |
| **assets** | Raw files (fonts, HTML, text) jaisi hain waisi rakhi jaati hain, file name se access hoti hain |
| **bin** | Compiled output: classes.dex aur APK |
| **libs** | Extra third-party .jar libraries |
| **res** | Resources: drawable-hdpi/mdpi/xhdpi, layout, menu, values (strings.xml, styles.xml) |
| **AndroidManifest.xml** | Project root me manifest file |
| **project.properties / proguard** | Target API setting / code shrinking aur obfuscation rules |

- **R.java / R class:** auto-generated class jo har resource ko unique integer ID deti hai (jaise `R.layout.activity_main`, `R.id.button1`, `R.string.app_name`). Tumhara code resources dhoondhne ke liye ise use karta hai.
- **res vs assets: res** ki files ko R me ID milti hai aur system unhe optimise karta hai; **assets** ki files chhui nahi jaati aur naam se kholi jaati hain.
- **Text strings.xml me kyun?** Isse doosri languages me translation aasaan hota hai aur hard-coded text se bachte hain.

### App kaise banta hai: source se APK tak

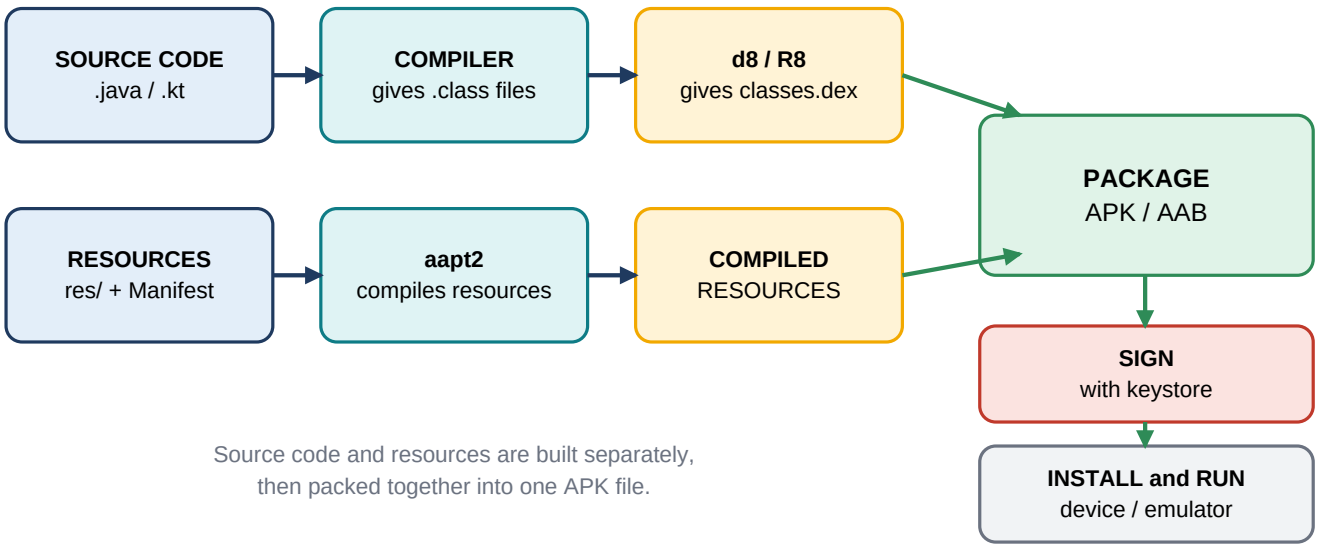



*Figure 6: Android build process*

- Compiler Java/Kotlin ko **.class** files me badalta hai; **d8** unhe **classes.dex** (Dalvik Executable) me convert karta hai jo ART chala sakta hai.
- **aapt2** resources aur manifest compile karta hai.
- Sab kuch ek **APK** (ya **AAB**, Android App Bundle) me package hota hai aur key se **sign** hota hai. Sirf signed app install ho sakta hai. Google Play ab naye apps ke liye AAB maangta hai.

### Android app ke chaar main components

| Component | Purpose | Example |
|---|---|---|
| **Activity** | UI wali ek screen | Login screen |
| **Service** | Background me bina screen ka kaam | Music chalna |
| **Broadcast Receiver** | System ya app ke messages par react karta hai | Battery low alert |
| **Content Provider** | Apps ke beech data share karta hai | Contacts ka data |

Ye chaaron components manifest me declare hote hain. Ye ek doosre ko start karne ke liye **Intents** (messages) use karte hain. Inka detail Unit II me hai.

## Topic 7: AndroidManifest.xml (Android Manifest file)

**Real-life example**

Har insaan ke paas **ID card** hota hai: naam, address, kya karne ki permission hai. **Manifest** app ka ID card hai. Android koi bhi code chalane se **pehle** ise padhta hai, ye jaanne ke liye ki app kya hai aur use kya chahiye.

**Definition**

**AndroidManifest.xml** har Android app ke root me ek zaroori XML file hai jo **Android system ko app ke baare me batati hai**: package name, components, permissions, hardware needs aur bahut kuch.

### Manifest kya karta hai

- App ka **package** naam batata hai (unique identity).
- **Har component declare** karta hai (activities, services, receivers, providers). Yahan declare na hone par component chal nahi sakta.
- App ko chahiye **permissions** (INTERNET, CAMERA, READ_CONTACTS) aur doosron ko ise use karne ke liye chahiye permissions ki list deta hai.
- **Intent filter** se batata hai ki kaunsi activity **launcher** (entry point) hai.
- Zaroori **hardware/software features** aur app ka icon, label aur theme declare karta hai.

**AndroidManifest.xml (example)**

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/Theme.HelloWorld">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

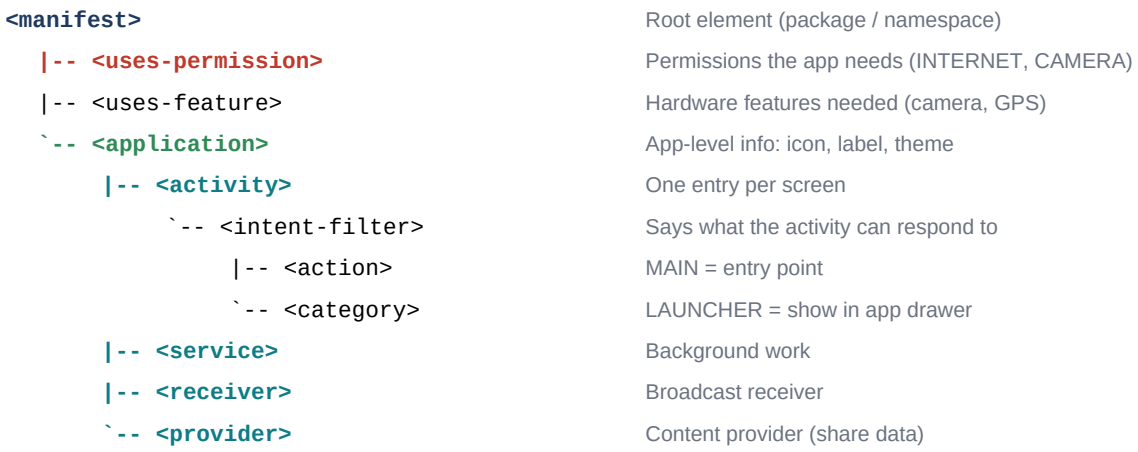



*Figure 7: Structure of AndroidManifest.xml*

| Element / attribute | Matlab |
|---|---|
| **<manifest>** | Root element. **package** attribute app ka unique naam hai. |
| **<uses-permission>** | Permission maangta hai, jaise android.permission.INTERNET |
| **<uses-feature>** | Zaroori hardware (camera, GPS); iske bina phones se Play Store app chhupa deta hai |
| **<application>** | Components ka container; attributes icon, label, theme, allowBackup |
| **<activity>** | Ek screen declare karta hai. **android:name** uski class hai (.MainActivity). |
| **<service>, <receiver>, <provider>** | Service, broadcast receiver aur content provider declare karte hain |
| **<intent-filter>** | Batata hai ki component kaunse intents handle kar sakta hai (action, category, data) |
| **MAIN + LAUNCHER** | MAIN = app ka entry point; LAUNCHER = app drawer me icon dikhao |
| **android:exported** | Doosre apps component ko start kar sakte hain ya nahi. Component me intent filter ho aur app Android 12 (API 31) ya usse upar target kare to zaroori hai. |

**Jaanna zaroori hai**

Purane projects aur zyada tar textbooks me **package** aur **<uses-sdk>** (minSdkVersion, targetSdkVersion) manifest me likhe hote hain. Naye Android Studio projects me package **namespace** ki tarah aur SDK versions **build.gradle** me set hote hain, isliye manifest me nahi hote. Exam me wahi likho jo textbook dikhati hai.

# PART B: Important Questions aur Model Answers (Hinglish)

Priority: **Very Likely** = BTEUP me sabse zyada aata hai. **Likely** = achha chance. **Possible** = aa sakta hai. Har answer exam format me hai: **Definition, Points, Diagram/Table, Conclusion**.

## Q1. Android kya hai? Uske features aur versions explain karo. — VERY LIKELY

**Definition / Parichay:** Android ek open-source, Linux-based mobile operating system hai jo Google aur Open Handset Alliance ne develop kiya hai. Ye smartphones, tablets, smart TVs, watches aur cars me use hota hai.

### Points

- **Open source aur free:** manufacturers ise customise kar sakte hain.
- **Rich user interface:** widgets, notifications, multitasking.
- **Connectivity:** Wi-Fi, Bluetooth, NFC, GPS, mobile data.
- **Storage:** SQLite, files, shared preferences. **Hardware access:** camera, sensors.
- **Development:** apps Java ya Kotlin me likhe aur Google Play par publish hote hain.
- **Security:** har app apne sandbox me chalta hai aur permissions maangta hai.
- **Versions:** Android 9 tak desserts ke naam par (Cupcake, Donut, Eclair, Froyo, Gingerbread, Honeycomb, Ice Cream Sandwich, Jelly Bean, KitKat, Lollipop, Marshmallow, Nougat, Oreo, Pie); Android 10 se numbers.

| Version | Name | API level |
|---|---|---|
| 1.5 | Cupcake | 3 |
| 4.4 | KitKat | 19 |
| 5.0 | Lollipop | 21 |
| 6.0 | Marshmallow | 23 |
| 8.0 | Oreo | 26 |
| 9 | Pie | 28 |
| 10 | (no name) | 29 |

**Conclusion:** Android sabse zyada use hone wala mobile platform hai kyunki ye open, flexible hai aur kai manufacturers ise support karte hain.

## Q2. Android ki architecture diagram ke saath explain karo. VERY LIKELY

**Definition / Parichay:** Android platform ek software stack hai jo layers me arranged hai. Har layer apne upar wali layer ko services deti hai.

### Points

- **Linux kernel:** foundation. Drivers, memory, process aur power management, aur security deta hai.
- **Hardware Abstraction Layer (HAL):** camera, audio, Bluetooth aur sensors jaise hardware ke liye standard interfaces.
- **Android Runtime (ART):** app ka DEX bytecode chalata hai; har app ka apna process aur ART instance; core Java libraries shamil (Android 5.0 se pehle Dalvik).
- **Native C/C++ libraries:** SQLite, OpenGL ES, SSL, libc aur media libraries.
- **Java API framework:** Activity Manager, View System, Content Providers, Resource Manager aur Notification Manager.
- **System apps:** phone, contacts, browser aur email jaise built-in apps; user apps isi level par chalte hain.

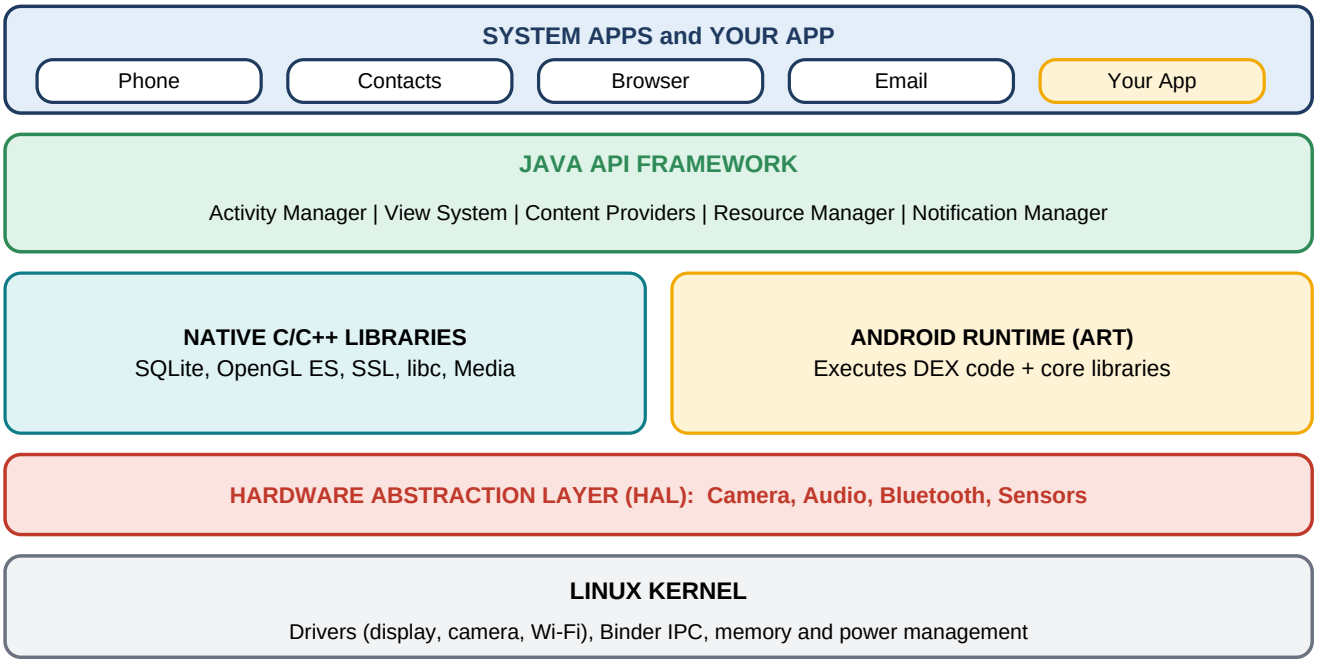



*Figure: Android software stack*

**Conclusion:** Layered design Android ko alag alag hardware par portable banata hai aur developers ko hardware details sambhale bina high-level APIs use karne deta hai.

## Q3. Android SDK kya hai? Uske components explain karo. VERY LIKELY

**Definition / Parichay:** Android SDK (Software Development Kit) libraries, tools, documentation aur sample code ka set hai jo Android applications build, test aur debug karne ke liye chahiye.

### Points

- **SDK Platforms:** har API level ki android.jar.
- **Platform-Tools:** adb (Android Debug Bridge) aur fastboot.
- **Build-Tools:** APK build aur sign karne ke liye aapt2, d8, apksigner, zipalign.
- **Emulator aur system images:** Android Virtual Devices (AVD) banane ke liye.
- **SDK Manager aur command-line tools:** packages download aur manage karte hain (sdkmanager, avdmanager).
- **Libraries aur tools:** AndroidX, Google Play services, debugging ke liye Logcat.

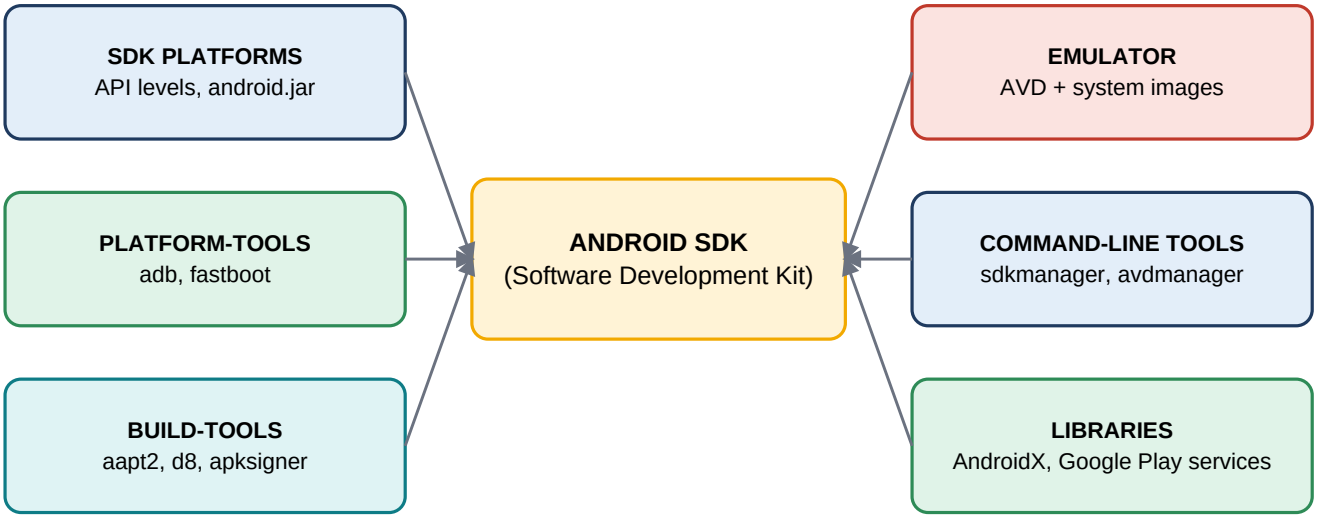



*Figure: Android SDK components*

**Conclusion:** SDK, JDK aur ek IDE ke saath mil kar Android apps develop karne ke liye sab kuch deta hai.

## Q4. Android Studio install karne ke steps explain karo. LIKELY

**Definition / Parichay:** Android Studio Android ka official Integrated Development Environment (IDE) hai. Isme code editor, SDK, emulator aur build tools shamil hain.

### Points

- System requirements dekho (64-bit OS, kam se kam 8 GB RAM jisme 16 GB recommended, kaafi disk space, virtualization on).
- developer.android.com/studio se Windows, macOS ya Linux ka installer download karo.
- Installer chalao, Android Studio aur Android Virtual Device selected rakho, folder chuno aur install karo.
- Android Studio shuru karo. Setup Wizard me Standard setup, theme chuno aur licences accept karo.
- Wizard SDK, platform-tools aur emulator download karta hai. Finish dabao.
- Baad me aur platforms ya tools ke liye Tools > SDK Manager use karo.

**Conclusion:** Installation ke baad naya project banakar emulator ya real phone par chalaya ja sakta hai.

## Q5. Android development ke liye Eclipse aur ADT plugin ki installation explain karo. LIKELY

**Definition / Parichay:** Android Studio se pehle Android apps Eclipse IDE me Android Developer Tools (ADT) plugin se banaye jaate the. Google ne ADT ka support 2016 me band kar diya, par ye procedure syllabus ka hissa hai.

### Points

- **JDK install karo** aur JAVA_HOME set karo.
- **Eclipse download aur unzip karo** (Eclipse IDE for Java Developers) aur chalao.
- **Android SDK download karo** aur SDK Manager se platform, platform-tools aur build-tools install karo.
- **ADT plugin install karo:** Help > Install New Software > Add, ADT update-site URL do, Developer Tools select karo, licence accept karo aur restart karo.
- **Configure karo:** Window > Preferences > Android > SDK Location set karo.
- Window > AVD Manager se **AVD banao**, phir File > New > Android Application Project se project banao.

**Conclusion:** Eclipse with ADT ab obsolete hai; Android Studio official aur recommended IDE hai.

## Q6. AVD kaise banate hain aur app ko emulator aur real device par kaise chalate hain?

LIKELY

**Definition / Parichay:** AVD (Android Virtual Device) computer par chalne wala virtual phone hai, jo real device ke bina apps test karne ke kaam aata hai.

### Points

- **AVD banao:** Tools > Device Manager > Create Device > hardware profile chuno > system image (API level) chuno > naam do > Finish.
- **Emulator par run:** Play button se AVD shuru karo, device list me use chuno aur Run dabao.
- **Real device:** Developer options on karo (Build number 7 baar tap), USB debugging on karo, USB se connect karo aur prompt allow karo.
- `adb devices` se check karo, phone chuno aur Run dabao.
- APK ko seedha `adb install app.apk` se bhi install kar sakte hain.

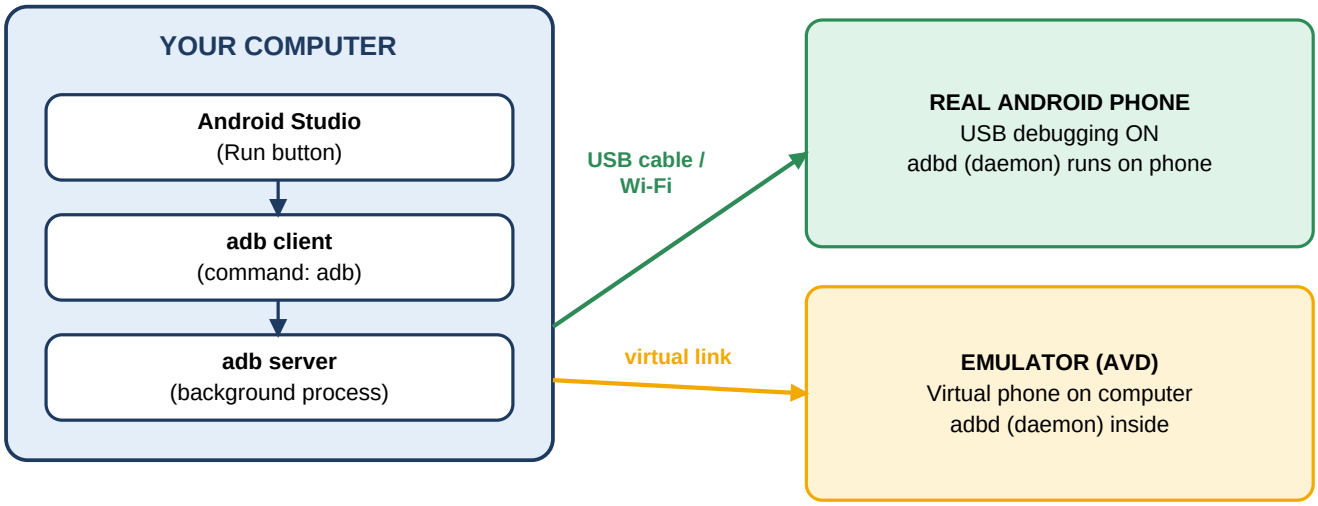



*Figure: IDE, adb and device*

**Conclusion:** Emulator alag alag devices test karne ke liye convenient hai, jabki real phone asli speed aur behaviour deta hai.

## Q7. Apna pehla Android application (Hello World) banane ke steps likho. VERY LIKELY

**Definition / Parichay:** Hello World application sabse simple Android app hai. Ye screen par 'Hello World!' text dikhata hai aur verify karta hai ki development environment sahi kaam kar raha hai.

### Points

- Android Studio kholo > **New Project** > **Empty Views Activity** chuno > Next.
- App name (HelloWorld), package name (com.example.helloworld), save location, language (Java/Kotlin) aur minimum SDK daalo > **Finish**.
- **Gradle** build khatam hone tak ruko.
- **MainActivity** me code hai; `onCreate()`, `setContentView(R.layout.activity_main)` ko call karke layout dikhata hai.
- **activity_main.xml** me `android:text="Hello World!"` wala ek **TextView** hai.
- Emulator ya phone chuno aur **Run** dabao.

**MainActivity.java**

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

**activity_main.xml (TextView)**

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

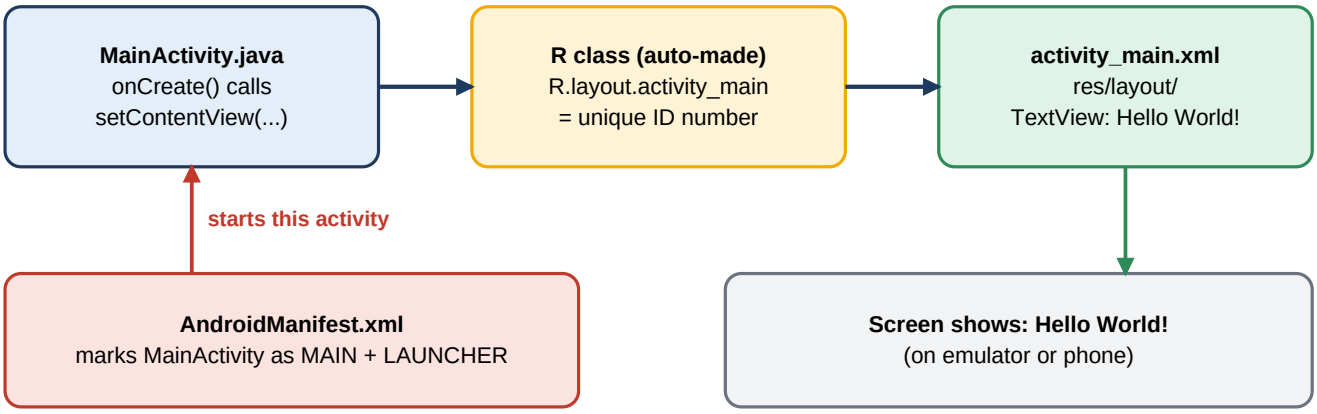



*Figure: How the parts of the app are connected*

**Conclusion:** Hello World app har Android app ke basic parts dikhata hai: ek activity, ek layout aur manifest.

## Q8. Android application ki anatomy (project structure) explain karo. VERY LIKELY

**Definition / Parichay:** Android application ki anatomy Android project ke folders aur files aur har ek ka purpose batati hai.

### Points

- **manifests:** AndroidManifest.xml system ko app ke baare me batati hai.
- **java (ya kotlin):** source code, jaise MainActivity.
- **res:** resources: drawable (images), layout (screens), mipmap (launcher icons), values (strings.xml, colors.xml, themes.xml).
- **Gradle Scripts:** build.gradle files jo applicationId, minSdk, targetSdk, versions aur dependencies set karti hain.
- **R class:** auto-generated class jo har resource ko unique ID deti hai.
- **Eclipse me:** src, gen (R.java), Android library, assets, bin, libs, res aur AndroidManifest.xml.

```
app                                  Main module of your app
  |-- manifests
       `-- AndroidManifest.xml       Declares components, permissions, app info
  |-- java
       `-- com.example.helloworld    Package name
            `-- MainActivity         Java / Kotlin source code (logic)
  `-- res
       |-- drawable                  Images and shapes
       |-- layout                    UI screens (activity_main.xml)
       |-- mipmap                    Launcher icons
       `-- values                    strings.xml, colors.xml, themes.xml
Gradle Scripts                       Build configuration
  |-- build.gradle (Project)         Settings for the whole project
  |-- build.gradle (Module: app)     App id, minSdk, targetSdk, dependencies
  |-- settings.gradle                Lists the modules
  |-- gradle.properties              Gradle options
  `-- local.properties               Local SDK path (not shared)
```

*Figure: Android Studio project structure*

**Conclusion:** Har file kahan rehti hai ye jaanne se Android app banana aur maintain karna aasaan hota hai.

## Q9. Android build process explain karo. R.java aur resources folder kya hain? LIKELY

**Definition / Parichay:** Build process source code aur resources ko install hone wali, signed APK ya AAB file me badalta hai.

### Points

- Java/Kotlin code **.class** files me compile hota hai aur **d8** use **classes.dex** me convert karta hai.
- **aapt2** resources aur manifest compile karta hai.
- Sab parts ek **APK** (ya **AAB**) me package hote hain aur keystore se sign hote hain.
- **R.java (R class):** auto-generated; har resource ko unique integer ID se map karta hai, jaise R.layout.activity_main, R.id.button1. Ise edit nahi karna chahiye.
- **res folder:** resources rakhta hai (drawable, layout, values, mipmap, menu). Text strings.xml me rakhne se translation aasaan hota hai.
- **assets folder:** raw files jo naam se access hoti hain, R me ID nahi hoti.

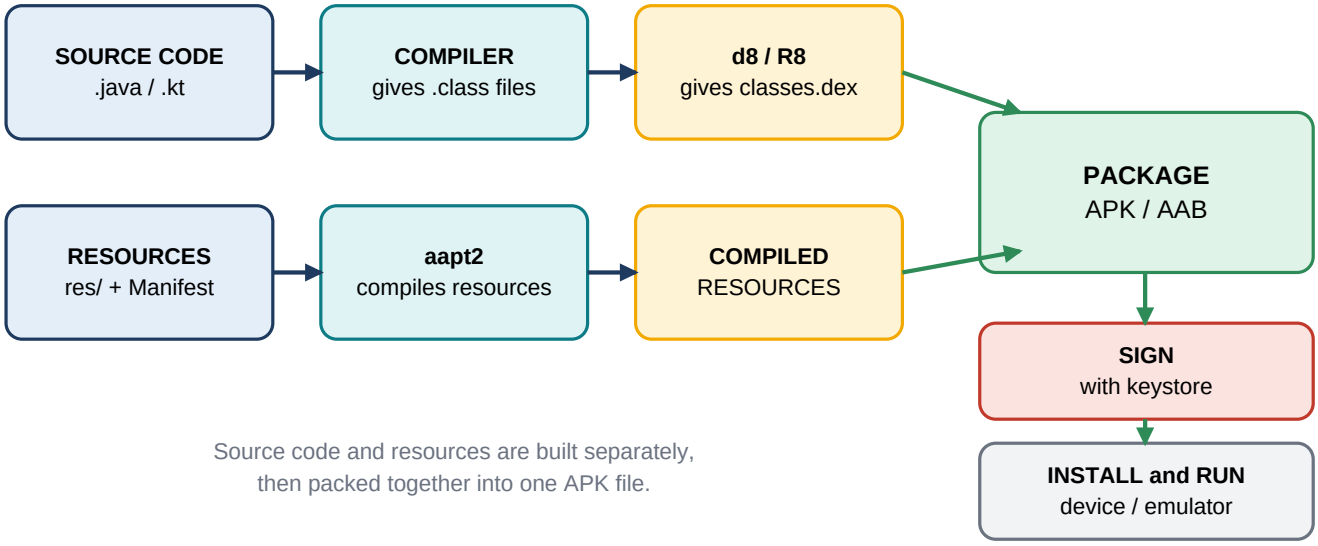



*Figure: Android build process*

**Conclusion:** Build system compiling, packaging aur signing automate kar deta hai, taaki developer app par focus kar sake.

## Q10. AndroidManifest.xml kya hai? Uska structure aur important elements explain karo.

VERY LIKELY

**Definition / Parichay:** AndroidManifest.xml har Android app ke root me ek zaroori XML file hai. Ye Android system ko app ke baare me batati hai, jo koi bhi code chalane se pehle ise padhta hai.

### Points

- **<manifest>:** root element; package attribute unique app name deta hai.
- **<uses-permission>:** app ko chahiye permissions, jaise INTERNET, CAMERA.
- **<uses-feature>:** app ko chahiye hardware features.
- **<application>:** app-level settings (icon, label, theme) aur ye components ko contain karta hai.
- **<activity>, <service>, <receiver>, <provider>:** chaaron components declare karte hain.
- **<intent-filter>** jisme action MAIN aur category LAUNCHER ho, launcher activity batata hai.

```
<manifest>                          Root element (package / namespace)
  |-- <uses-permission>             Permissions the app needs (INTERNET, CAMERA)
  |-- <uses-feature>                Hardware features needed (camera, GPS)
  `-- <application>                 App-level info: icon, label, theme
      |-- <activity>                One entry per screen
          `-- <intent-filter>       Says what the activity can respond to
              |-- <action>          MAIN = entry point
              `-- <category>        LAUNCHER = show in app drawer
      |-- <service>                 Background work
      |-- <receiver>                Broadcast receiver
      `-- <provider>                Content provider (share data)
```

*Figure: Structure of AndroidManifest.xml*

**Example manifest**

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld">
    <uses-permission android:name="android.permission.INTERNET" />
    <application android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

**Conclusion:** Sahi manifest ke bina app install ya run nahi ho sakta, kyunki system ko uske components ya permissions pata nahi chalenge.

## Q11. Android application ke components explain karo. LIKELY

**Definition / Parichay:** Android application chaar tarah ke components se bani hoti hai. Har ek manifest me declare hota hai aur system use start kar sakta hai.

### Points

- **Activity:** UI wali ek screen, jaise login screen.
- **Service:** background me bina UI ke chalti hai, jaise music bajana.
- **Broadcast Receiver:** system-wide messages par react karta hai, jaise battery low.
- **Content Provider:** apps ke beech data share karta hai, jaise contacts.
- **Intents** wo messages hain jinse components start hote hain.

| Component | Purpose | Example |
|---|---|---|
| Activity | UI wali ek screen | Login screen |
| Service | Background kaam | Music chalna |
| Broadcast Receiver | Messages par react | Battery low |
| Content Provider | Data share | Contacts |

**Conclusion:** Ye components app ko modular banate hain, aur har ek ko intents ke zariye doosre apps reuse kar sakte hain.

## Q12. Dalvik aur ART me difference likho. APK aur AAB me kya difference hai? POSSIBLE

**Definition / Parichay:** Dalvik aur ART wo runtimes hain jo Android apps ko execute karte hain. ART ne Android 5.0 (API level 21) me Dalvik ki jagah default runtime ban gaya.

### Points

- Dono apps ko **DEX** (Dalvik Executable) format me chalate hain.
- **Dalvik** code ko run time par compile karta tha (just-in-time), jisme CPU aur battery zyada lagti thi. **ART** install ke time hi code compile karta hai (ahead-of-time), isliye apps tez shuru hote hain; naye versions AOT, JIT aur profile-guided compilation ko mila kar use karte hain.
- **APK** (Android Package) install hone wali file hai jisme sab devices ke liye sab kuch hota hai. **AAB** (Android App Bundle) publishing format hai jo Google Play par upload hota hai, jo phir har device ke liye optimised APKs banata hai; Google Play naye apps ke liye AAB maangta hai.

| Point | Dalvik | ART |
|---|---|---|
| Compilation | Just-in-time (run time par) | Install par ahead-of-time (naye versions me JIT, profile-guided bhi) |
| App start | Slow | Tez |

| Point | APK | AAB |
|---|---|---|
| Purpose | Install hone wali file | Google Play ke liye publishing format |
| Seedha install | Haan | Nahi (Google Play isse APKs banata hai) |
| Device par size | Bada (sab resources) | Chhota (sirf wahi jo device ko chahiye) |

**Conclusion:** ART Dalvik se behtar performance deta hai, aur AAB ek universal APK se chhota download deta hai.

# Quick Revision Sheet (exam se pehle raat ko padho)

| Term | Ye yaad rakho |
|---|---|
| Android | Google + OHA ka open-source, Linux-based OS. Pehla phone HTC Dream (2008) |
| Architecture | Linux kernel > HAL > ART + C/C++ libraries > Java API framework > System apps |
| ART / Dalvik | ART Android 5.0 (API 21) se default; DEX chalata hai |
| Android SDK | Platforms, Platform-Tools (adb), Build-Tools, Emulator, Command-line tools |
| API level | Har Android version ka number. minSdk, targetSdk, compileSdk |
| Android Studio | Official IDE. Eclipse ADT support 2016 me khatam |
| Eclipse setup | JDK > Eclipse > SDK > ADT plugin > SDK path > AVD |
| AVD | Virtual phone. Tools > Device Manager > Create Device |
| Real phone | Build number 7 baar tap > USB debugging ON > adb devices |
| adb | Android Debug Bridge. adb install app.apk |
| First app | Empty Views Activity > Name, Package, Language, Min SDK > Finish > Run |
| onCreate() | Activity banne par call hota hai; setContentView() call karta hai |
| Project folders | manifests, java, res (drawable, layout, mipmap, values), Gradle Scripts |
| R class | Resources ki auto-generated IDs. Kabhi edit mat karo |
| res vs assets | res ko IDs milti hain; assets raw files hain jo naam se khulti hain |
| Build | Source > .class > d8 > classes.dex; resources > aapt2; APK/AAB package; sign |
| 4 components | Activity, Service, Broadcast Receiver, Content Provider |
| Manifest | Root XML. package, uses-permission, application, activity, intent-filter (MAIN, LAUNCHER) |
| android:exported | Android 12+ (API 31) target par intent-filter ke saath zaroori |

**Yaad rakhne ke tricks**

**Architecture (neeche se upar): L-H-R-J-S** = Linux, HAL, Runtime (aur libraries), Java framework, System apps.
**4 components: A-S-B-C** = Activity, Service, Broadcast receiver, Content provider.
**res folders: D-L-M-V** = drawable, layout, mipmap, values.
**Manifest launcher: MAIN** = entry point, **LAUNCHER** = home screen par icon.

# Common Mistakes jo Students Karte Hain

- Eclipse ko official IDE likhna. Ab **Android Studio** official hai; Eclipse ADT support 2016 me khatam hua.
- Ye bhool jana ki manifest **saare components declare** karta hai. Manifest me na ho to component chalega nahi.
- **R.java** ko haath se edit karna. Ye auto-generated hai.
- **res** (IDs wale resources) aur **assets** (raw files) ko mix karna.
- Architecture diagram aur project tree na banana. Inme marks hote hain.

## Self-Test (bina dekhe likho, har question 5 marks)

- Android ki architecture draw karke explain karo.
- Android SDK kya hai? Koi chaar components likho.
- Hello World application banane aur chalane ke steps likho.
- AndroidManifest.xml kya hai? Koi chaar elements likho.
- Android application ke chaar components ek ek example ke saath likho.

## Self-Test Answers (likhne ke baad check karo)

**1.** Neeche se layers: Linux kernel, HAL, Android Runtime (ART) aur native C/C++ libraries, Java API framework, System apps. Har ek ko ek line me samjhao. Figure 1 banao.

**2.** SDK apps build, test aur debug karne ke liye libraries, tools, documentation aur samples ka set hai. Components: SDK Platforms, Platform-Tools (adb), Build-Tools, Emulator aur system images, command-line tools, libraries.

**3.** New Project > Empty Views Activity > name, package, language, minimum SDK daalo > Finish > Gradle ruko > MainActivity aur activity_main.xml edit karo > emulator ya phone chuno > Run.

**4.** Manifest wo XML file hai jo Android system ko app ke baare me batati hai. Elements: manifest, uses-permission, uses-feature, application, activity, service, receiver, provider, intent-filter.

**5.** Activity (login screen), Service (music chalna), Broadcast Receiver (battery low alert), Content Provider (contacts ka data).

> **Unit I complete!**
> Agar 5 me se 4 ya zyada sahi hue to **Unit II** par jao. Nahi to Part A me jo topics miss hue unhe dobara padho.